

Algorithmes et Pensée Computationnelle

Architecture des ordinateurs - Exercices avancés

Le but de cette séance est de comprendre le fonctionnement d'un ordinateur. La série d'exercices sera axée sur les conversions en base binaire, décimale ou hexadécimale, ainsi que des opérations de base en suivant le modèle Von Neumann.

Cette feuille d'exercices avancés vous permettra d'approfondir vos connaissances des notions vues en cours.

Question 1: (🕒 20 minutes) Conversion et addition :

Effectuer les opérations suivantes :

1. $111101_{(2)} + 110_{(2)} = \dots_{(10)}$
2. $111111_{(2)} + 000001_{(2)} = \dots_{(10)}$
3. $127_{(10)} + ABC_{(16)} = \dots_{(10)}$

💡 Conseil

Calculez à l'aide du tableau d'addition binaire ci-dessus ou une autre méthode que vous préférez. Additionnez les nombres lorsqu'ils sont dans la même base puis convertissez les dans la base souhaitée.

Exemple de conversion : $1001000_{(2)} = 72_{(10)}$

Base ₍₂₎	1	0	0	1	0	0	0	
Position ₍₂₎	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	↓	↓	↓	↓	↓	↓	↓	
Équivalent ₍₁₀₎	1×2^6	0×2^5	0×2^4	1×2^3	0×2^2	0×2^1	0×2^0	
Valeurs ₍₁₀₎	64	0	0	8	0	0	0	= 72

Rappel : Les valeurs en hexadécimale (base 16)

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Exemple de conversion : $3BF_{(16)} = 959_{(10)}$

Base ₍₁₆₎	3	B	F	
Valeurs ₍₁₆₎	3	11	15	
Position ₍₁₆₎	16^2	16^1	16^0	
	↓	↓	↓	
Équivalent ₍₁₀₎	3×16^2	11×16^1	15×16^0	
Valeurs ₍₁₀₎	768	176	15	= 959

Exemple de conversion : $123_{(10)} = \dots_{(8)}$

$$8^0 = 1 < 123 \quad 8^1 = 8 < 123 \quad 8^2 = 64 < 123 \quad 8^3 = 512 > 123$$

$$123/8 = 15 \text{ avec un reste de } 3$$

$$15/8 = 1 \text{ avec un reste de } 7$$

$$1/8 = 0 \text{ avec un reste de } 1$$

$$123_{(10)} = 173_{(8)}$$

Question 2: (🕒 20 minutes) Conversion et soustraction :

Effectuer les opérations suivantes :

1. $101010_{(2)} - 010101_{(2)} = \dots_{(10)}$
2. $64_{(10)} - 001000_{(2)} = \dots_{(10)}$
3. $FFF_{(16)} - 127_{(10)} = \dots_{(2)}$

💡 Conseil

Calculez à l'aide du tableau de soustraction binaire ci-dessus ou une autre méthode que vous préférez.

Soustrayez les nombres lorsqu'ils sont dans la même base puis convertissez-les dans la base souhaitée.

Exemple de conversion : $1001000_{(2)} = 72_{(10)}$

Base ₍₂₎	1	0	0	1	0	0	0	
Position ₍₂₎	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	↓	↓	↓	↓	↓	↓	↓	
Équivalent ₍₁₀₎	1×2^6	0×2^5	0×2^4	1×2^3	0×2^2	0×2^1	0×2^0	
Valeurs ₍₁₀₎	64	0	0	8	0	0	0	= 72

Rappel : Les valeurs en hexadécimale (base 16)

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Exemple de conversion : $123_{(10)} = \dots_{(2)}$

$$\begin{array}{llllll}
 2^0 = 1 < 123 & 2^1 = 2 < 123 & 2^2 = 4 < 123 & 2^3 = 8 < 123 & 2^4 = 16 < 123 \\
 2^5 = 32 < 123 & 2^6 = 64 < 123 & 2^7 = 128 > 123 & & &
 \end{array}$$

 $123/2 = 61$ avec un reste de **1** (premier chiffre en partant de droite)

 $61/2 = 30$ avec un reste de **1**
 $30/2 = 15$ avec un reste de **0**
 $15/2 = 7$ avec un reste de **1**
 $7/2 = 3$ avec un reste de **1**
 $3/2 = 1$ avec un reste de **1**
 $1/2 = 0$ avec un reste de **1** (dernier chiffre en partant de droite)

 $123_{(10)} = \mathbf{1111011}_{(2)}$